

Python

Рекурсия



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

-  Документация
-  Аннотации типов
-  Аннотации для базовых типов
-  Аннотации для структур данных
-  Аннотации Any, Union и Optional
-  Передача неизменяемых и изменяемых объектов

План занятия

- стек вызовов
- Рекурсия
- Рекурсия или итерация?
- Задачи, решаемые рекурсией



ОСНОВНОЙ БЛОК





Стек вызовов



Стек вызовов (call stack)

Это структура данных, которая используется для управления порядком выполнения функций в программе. Когда функция вызывается, её данные помещаются в стек, а после завершения работы – удаляются из него

Основные принципы работы стека ВЫЗОВОВ



LIFO (Last In, First Out) – последняя вызванная функция выполняется первой



Каждый вызов функции добавляет **новый фрейм** в стек вызовов



Когда функция завершается (return), её фрейм удаляется из стека

Пример вложенных вызовов функций

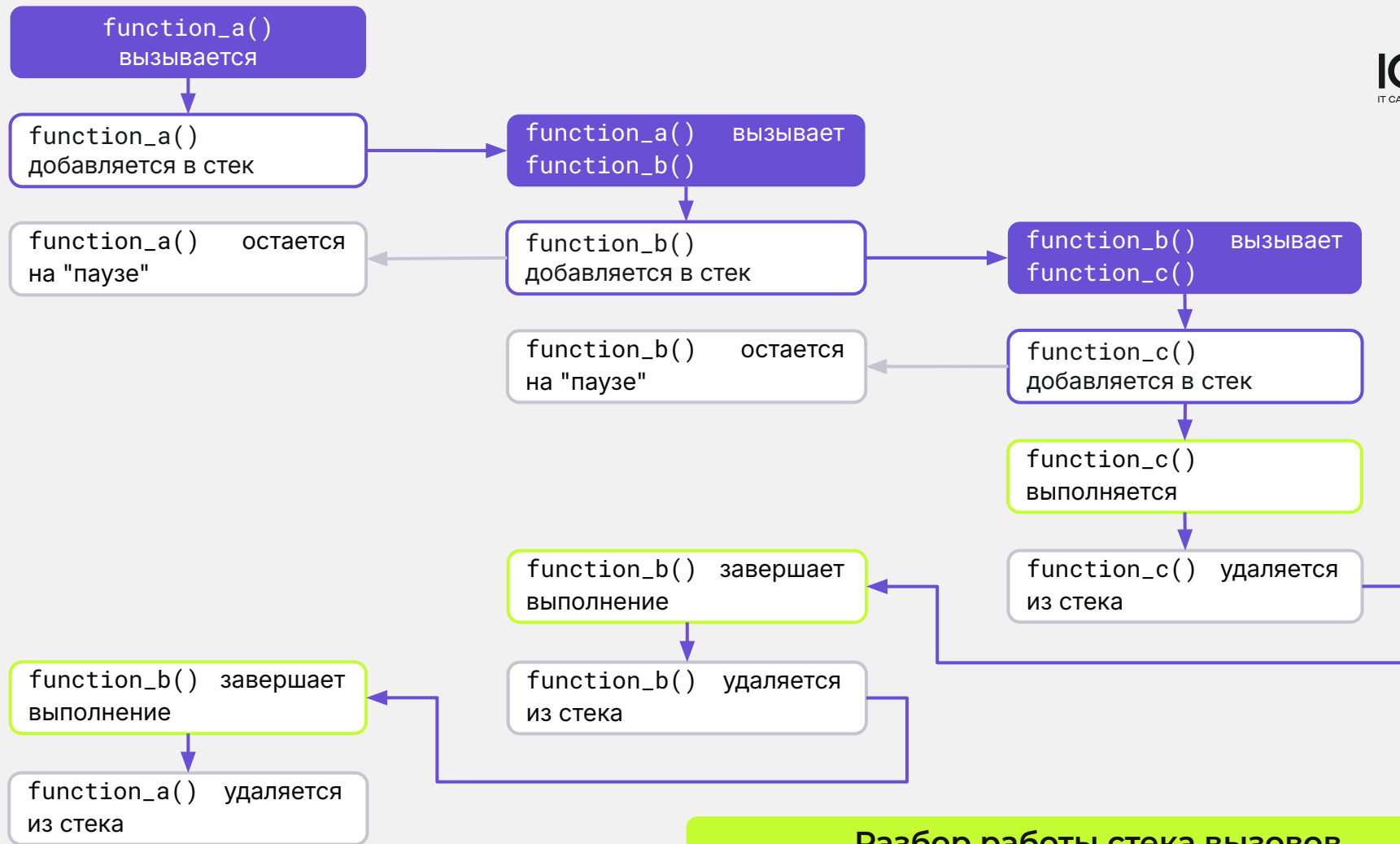


```
def function_a():  
    print("Начало A")  
    function_b()  
    print("Конец A")
```

```
def function_b():  
    print("Начало B")  
    function_c()  
    print("Конец B")
```

```
def function_c():  
    print("Начало C")  
    print("Конец C")
```

```
function_a()
```





ВОПРОСЫ





ЗАДАНИЕ





Выберите верный вариант ответа

Какой принцип используется в стеке вызовов?

- a. FIFO (First In, First Out)
- b. FILU (First In, Last Used)
- c. FILO (First In, Last Out)
- d. LIFO (Last In, First Out)



Выберите верный вариант ответа

Какой принцип используется в стеке вызовов?

- a. FIFO (First In, First Out)
- b. FILU (First In, Last Used)
- c. FILO (First In, Last Out)
- d. LIFO (Last In, First Out)



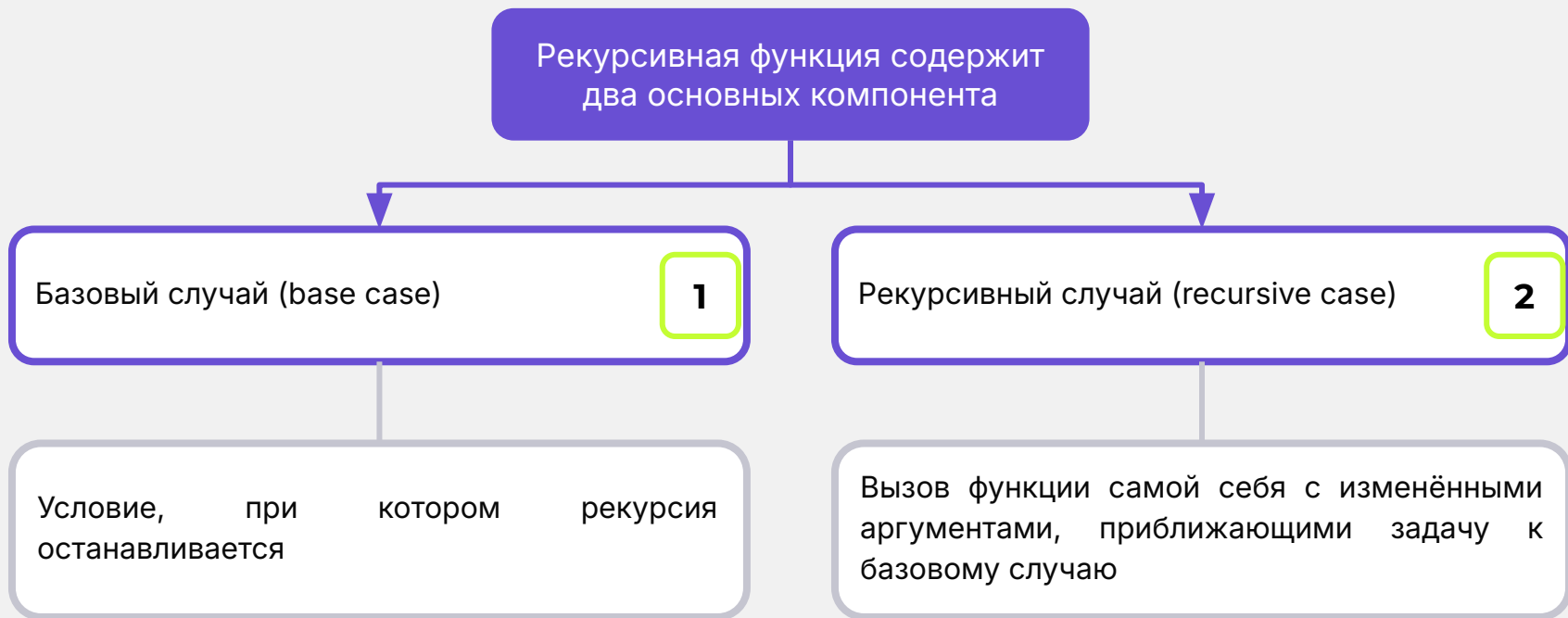
Рекурсия



Рекурсия

Это подход в программировании, при котором функция вызывает саму себя. Такой подход позволяет разбить сложную задачу на меньшие подзадачи, которые решаются тем же алгоритмом

Принцип работы рекурсивной функции



Пример 1: Простейшая рекурсивная функция (вывод чисел от n до 1)



```
def countdown(n: int) -> None:
    if n <= 0: # Базовый случай
        print("Конец!")
        return
    print(n)
    countdown(n - 1) # Рекурсивный случай
```

```
countdown(5)
```

Часто тот же процесс можно реализовать с помощью **цикла**:

```
def countdown_iterative(n: int) -> None:
    while n > 0:
        print(n)
        n -= 1
    print("Конец!")
countdown_iterative(5)
```



RecursionError

Когда функция вызывает саму себя (рекурсия), каждый новый вызов создаёт новый фрейм в стеке, что может привести к переполнению стека

Пример бесконечной рекурсии (не делать так!)



Код

```
def infinite_recursion():  
    infinite_recursion()  
  
infinite_recursion()
```

Пояснения

Этот код вызовет ошибку `RecursionError: maximum recursion depth exceeded`, так как стек вызовов заполнится и программа завершится сбоем

Пример ошибки



RecursionError Traceback (most recent call last)

Cell In[66], line 5

```
1 def infinite_recursion():  
2     infinite_recursion()  
----> 5 infinite_recursion()
```

Cell In[66], line 2, in infinite_recursion()

```
1 def infinite_recursion():  
----> 2     infinite_recursion()
```

Cell In[66], line 2, in infinite_recursion()

```
1 def infinite_recursion():  
----> 2     infinite_recursion()
```

[... skipping similar frames: infinite_recursion at line 2 (2971 times)]

Cell In[66], line 2, in infinite_recursion()

```
1 def infinite_recursion():  
----> 2     infinite_recursion()
```

RecursionError: maximum recursion depth exceeded

Пример рекурсивной функции: Факториал числа



```
def factorial(n: int) -> int:
    if n == 0 or n == 1: # Базовый случай
        return 1
    return n * factorial(n - 1) # Рекурсивный случай

print(factorial(5))
```

Факториал числа $n!$ – это произведение всех чисел от 1 до n .



Бинарный поиск

Это эффективный алгоритм поиска элемента в отсортированном списке

Пример рекурсивной функции: Бинарный поиск



```
def binary_search(arr: list[int], target: int, left: int, right: int) -> int:
    if left > right: # Базовый случай: элемент не найден
        return -1
    mid = (left + right) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search(arr, target, mid + 1, right) # Поиск в правой части
    else:
        return binary_search(arr, target, left, mid - 1) # Поиск в левой части

array = [1, 3, 5, 7, 9, 11, 13]
print(binary_search(array, 5, 0, len(array) - 1))
print(binary_search(array, 13, 0, len(array) - 1))
print(binary_search(array, 8, 0, len(array) - 1))
```



Хвостовая рекурсия

Это особый вид рекурсии, в котором рекурсивный вызов является последней операцией функции перед возвратом результата

Разница между обычной и хвостовой рекурсией



Обычная рекурсия

```
def factorial(n: int) -> int:
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

Хвостовая рекурсия

```
def factorial_tail(n: int, accumulator:
int = 1) -> int:
    if n == 0 or n == 1:
        return accumulator
    return factorial_tail(n - 1, n *
accumulator)

print(factorial_tail(5))
```

Использование хвостовой рекурсии



В Python нет встроенной оптимизации хвостовой рекурсии, которая **заменяет хвостовой рекурсивный вызов на повторное использование текущего кадра стека**. Поэтому рекомендуется **заменять хвостовую рекурсию на итерацию**, чтобы избежать переполнения стека при большом количестве рекурсивных вызовов.

Итеративный вариант факториала



```
def factorial_iterative(n: int) -> int:
    accumulator = 1
    while n > 1:
        accumulator *= n
        n -= 1
    return accumulator

print(factorial_iterative(5))
```



ВОПРОСЫ





ЗАДАНИЕ





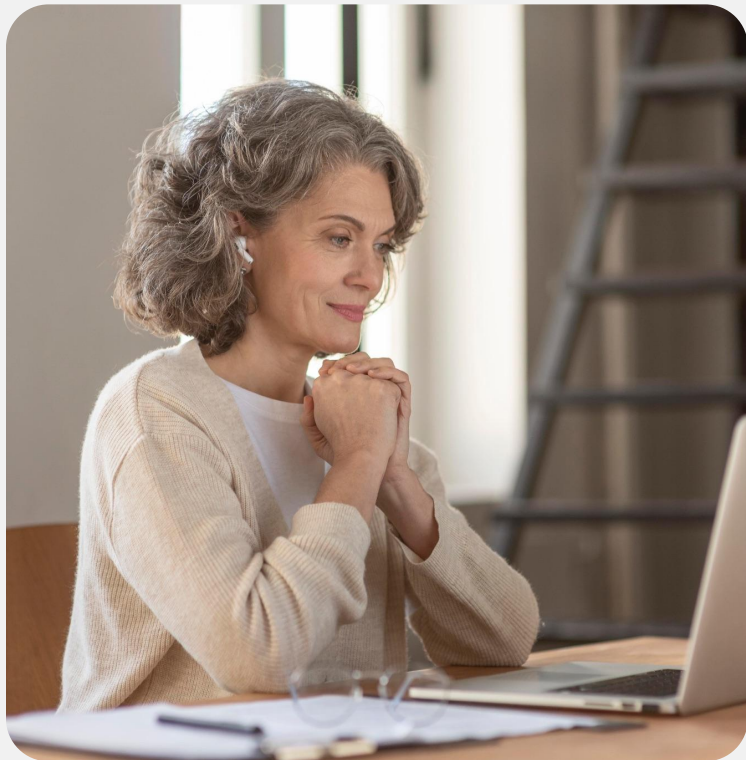
Выберите верный вариант ответа

Что произойдёт при выполнении следующего кода?

```
def infinite():  
    return infinite()
```

```
infinite()
```

- a. Бесконечный цикл
- b. Ошибка RecursionError
- c. Программа завершится без ошибок
- d. Ошибка SyntaxError



Выберите верный вариант ответа

Что произойдёт при выполнении следующего кода?

```
def infinite():  
    return infinite()
```

```
infinite()
```

- a. Бесконечный цикл
- b. Ошибка RecursionError**
- c. Программа завершится без ошибок
- d. Ошибка SyntaxError



Ответьте на вопрос

У вас есть две функции. Чем они отличаются и что будет выведено в результате выполнения?

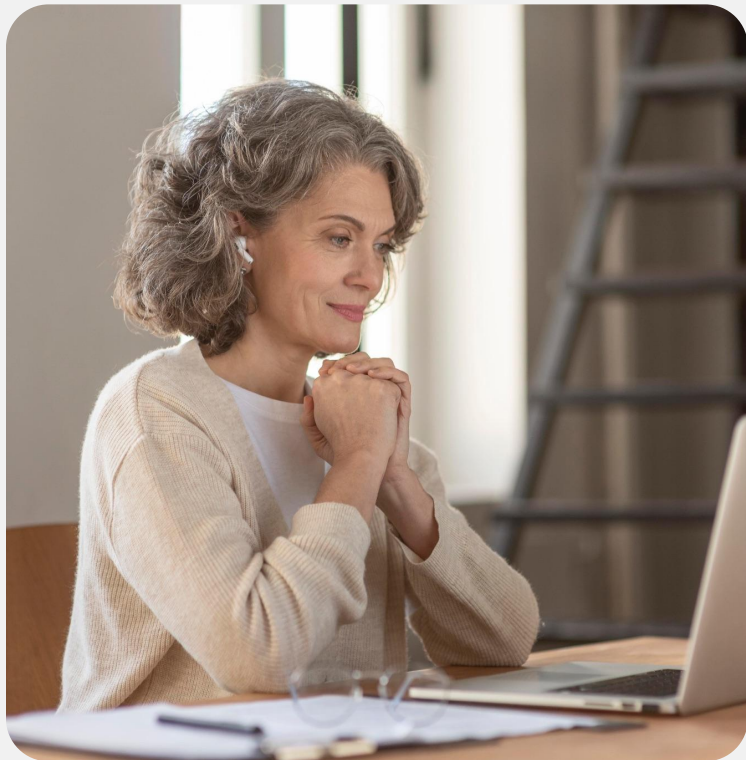
Функция 1

```
def print_numbers(n):  
    if n == 0:  
        return  
    print(n)  
    print_numbers(n - 1)
```

Функция 2

```
def print_nums(n):  
    if n == 0:  
        return  
    print_nums(n - 1)  
    print(n)
```

Ответьте на вопрос



У вас есть две функции. Чем они отличаются и что будет выведено в результате выполнения?

Функция 1

```
def print_numbers(n):  
    if n == 0:  
        return  
    print(n)  
    print_numbers(n - 1)
```

Функция 2

```
def print_nums(n):  
    if n == 0:  
        return  
    print_nums(n - 1)  
    print(n)
```

Ответ: обычная и хвостовая рекурсия. Разный порядок вывода чисел.



ВОПРОСЫ





Рекурсия или итерация?

Рекурсия и итерация



Это два подхода к решению задач, которые могут быть взаимозаменяемыми в некоторых случаях. Однако выбор между ними зависит от производительности, читабельности кода и ограничений памяти

Преимущества и недостатки рекурсии

Плюсы

- Позволяет разделить сложную задачу на подзадачи
- Упрощает чтение кода (например, обход деревьев и графов)
- Некоторые алгоритмы легче выразить рекурсивно (например, поиск пути в лабиринте)

Минусы

- Может привести к переполнению стека (`RecursionError`)
- Медленнее итеративных решений из-за затрат на вызовы функций
- Использует дополнительную память (каждый вызов создаёт новый стековый фрейм)

Когда использовать рекурсию?



Когда задача делится на подзадачи (разбиение на более мелкие части)



Для работы с деревьями, графами и вложенными структурами



Когда важна выразительность кода (код читается легче, чем итеративная версия)

Когда использовать итерацию?



Когда рекурсия создаёт избыточную нагрузку на стек вызовов



Когда можно решить задачу без хранения промежуточных вызовов



Когда важна эффективность по памяти и скорости



ВОПРОСЫ





Задачи, решаемые рекурсией

Важно



Некоторые задачи сложно или невозможно эффективно решить без использования рекурсии. Это связано с необходимостью **разбиения задачи на более мелкие подзадачи**, а также с работой со сложными вложенными структурами



Дерево

Это иерархическая структура данных, где каждый узел может иметь несколько дочерних узлов

1. Обход дерева (DFS - поиск в глубину)

Задачи, решаемые рекурсией



Рекурсивный подход позволяет обойти все узлы, последовательно углубляясь в каждый из них



Граф

Это структура, состоящая из узлов и рёбер, соединяющих их

2. Обход графа (DFS, рекурсивный поиск путей)

Задачи, решаемые рекурсией



Использование рекурсии в глубинном поиске позволяет эффективно находить пути между вершинами

3. Разбор выражений и синтаксический анализ

Задачи, решаемые рекурсией



При обработке вложенных математических выражений или языковых конструкций удобно использовать рекурсию, так как каждое подвыражение может рассматриваться как самостоятельная подзадача.

4. Ханойские башни

Задачи, решаемые рекурсией



Задача, требующая последовательного перемещения дисков между тремя стержнями с сохранением их порядка. Разбивается на две меньшие аналогичные задачи, что делает рекурсию естественным решением.

5. Разбиение на подмножества и комбинаторика

Задачи, решаемые рекурсией



Задачи, связанные с генерацией всех возможных комбинаций, перестановок и подмножеств множества, решаются с разбиением на подзадачи: выбор одного элемента и рекурсивное построение оставшейся структуры.



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА





Обратный вывод строки

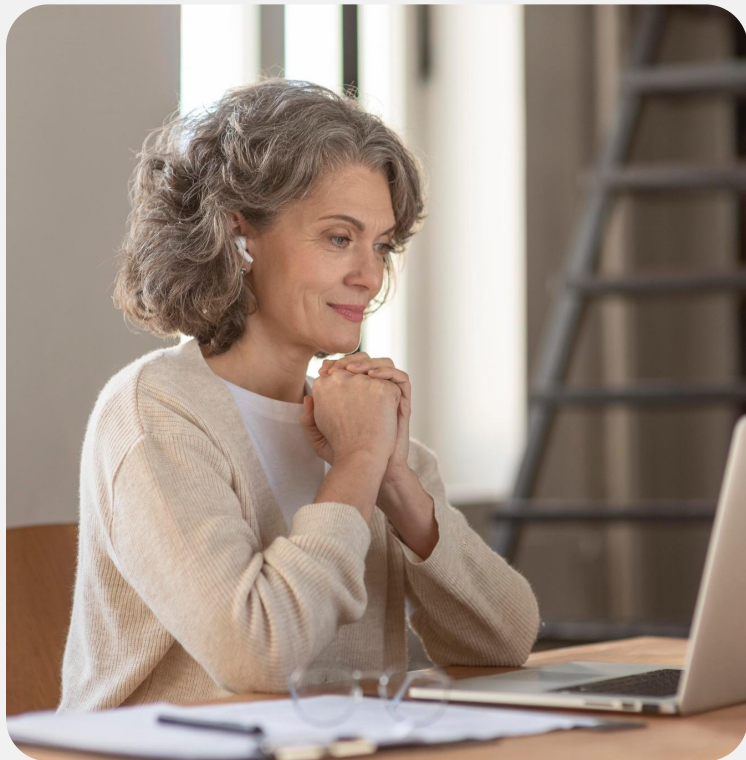
Напишите рекурсивную функцию, которая выводит строку в обратном порядке.

Данные

```
text = "hello"
```

Пример вывода

```
olleh
```



Подсчёт определённых слов

Напишите рекурсивную функцию, которая подсчитывает количество вхождений определённого слова во вложенной структуре (список списков строк).

Данные

```
nested_sentences = [
    ["Python is great", "I love Python"],
    ["Python is powerful", ["Python is everywhere",
    "Learn Python"]],
    "Coding in Python is fun"
]
word = "Python"
```

Пример вывода

Количество вхождений: 6



ДОМАШНЕЕ ЗАДАНИЕ



Домашнее задание

Сумма цифр числа

Напишите рекурсивную функцию, которая находит сумму всех цифр числа

Данные:

num = 43197

Пример вывода:

24

Домашнее задание

Сумма вложенных чисел

Напишите рекурсивную функцию, которая суммирует все числа во вложенных списках.

Данные:

```
nested_numbers = [1, [2, 3], [4, [5, 6]], 7]
```

Пример вывода:

28

Заключение

